Quiz 3 · Git & GitHub for collaboration

Friday, June 5, 2026

Today: Friday June 5

Before class

- **Perusall Assignment**: *DC* §9.1–9.2

Plan for today

- **Quiz 3** (first 20 minutes)
- Why version control
- Git vs. GitHub, push/pull
- Demo
- Weekend: get Git working + set up the team repo

Helpful links

- Syllabus · AI policy · Schedule · Happy Git with R · *DC* Ch. 9 · GitHub

Quiz 3

Quiz 3: 20 minutes

- Closed-book, closed-notes, closed-internet, closed-AI.
- Covers **weeks 1–3**: data frames and ggplot2, plus this week's wrangling, the six verbs (`filter`, `select`, `mutate`, `arrange`, `group_by`, `summarise`) and grouped summaries.

When you finish, turn in your quiz up front.

Why version control?

The naive approach

We've all probably done some version of the following:

```
analysis.R
analysis_v2.R
analysis_final.R
analysis_final_REAL.R
analysis_final_REAL_usethis_one.R
```

It sort of works, until you can't remember which is which, or two people email versions back and forth and the changes get lost.

> Tip
>
> **Version control** is the tool built for exactly this problem. You can think of it as a time machine for your project.

Version control overview

A version control system lets you:

- **Record the full history** of every file, not just the latest copy
- **Revert** to any earlier state if something breaks
- **Work with a team** without stepping on each other's files
- **Draft freely** without touching the "good" version until you're ready

Instead of saving a new copy of every file each time, Git stores just what changed, so it can reconstruct any earlier version on demand.

Helpful for solo work too

Even if you're working solo, version control is worthwhile; you might be:

- Working across **two computers** (laptop + lab machine)
- Coming back to a project **months later** and needing its history
- An **insurance policy** if your laptop is lost or dies

> **Note**
>
> Paired with **literate programming** (Quarto), version control is what makes an analysis *reproducible*: a complete record from raw data to final result.

Git vs. GitHub

The distinction

People often use "git" as a catch-all, but there's actually two pieces:

Table 1: Git is the tool; GitHub is one place to keep it.

	Git	**GitHub**
What	The version-control *tool*	A *website* that hosts repos
Where	On your computer (local)	In the cloud (remote)
Job	Tracks changes, makes commits	Stores & shares the project

A **repo** (repository) is just a folder Git is watching. It may include your `.qmd` files, data, images, etc. You keep a **local** copy on your computer and a **remote** copy on GitHub, and sync between them.

Daily workflow

Four workflow steps

Most days, the whole workflow fits in four steps:

1. **Edit & save** your files, as you have been doing already.
2. **Stage + Commit**: take a snapshot with a short message saying what changed.
3. **Pull**: grab any changes from GitHub *before* you send yours.
4. **Push**: send your commits up to GitHub.

> **Tip**
>
> **Commit early and often.** Each commit is a labeled stop on the time machine. Lots of small, well-described commits are much more clear and easy-to-follow than one giant mystery commit.

Write commit messages your future self can read

The message is how you (and your teammates) navigate the history later without having to read through all of the various versions of your code.

```
# Helpful
"Add body-mass-by-species summary table"
"Fix na.rm bug in flipper-length means"

# Useless
"stuff"
"asdf"
"changes"
```

> **Note**
>
> If you commit with a vague message, you'll have no idea what's there without opening it.

Where this lives: the RStudio Git tab

Once Git is configured, you can get away with barely leaving RStudio. A **Git** tab appears in the top-right pane with:

- A list of changed files, each with a **checkbox** to *stage* it
- A **Commit** button (opens a box for your message)
- **Pull** (down arrow) and **Push** (up arrow) buttons

This means we won't need to mess with the command line for our project work.

Live demo

Watch this once

I'll run the full loop on the projector. **Don't follow along yet**; just watch how the pieces connect. You'll do it yourself this weekend.

1. Create a new repo on **GitHub** (with a README).
2. In RStudio: **New Project > Version Control > Git**, paste the repo URL.
3. Edit the README, **save**.
4. Git tab: **stage** the change, **Commit** with a message, then **Push**.

5. Refresh the GitHub page, the change is there.
6. Edit a file *on GitHub*, then **Pull** in RStudio to bring it down.

> **Tip**
>
> Notice there was nothing about data analysis here. Git sits *underneath* the work you already do, simply creating snapshots and syncing the files.

Git + Teamwork

For your project team:

- One shared repo on GitHub; everyone has a local clone.
- Each person edits, commits, and pushes their work.
- **Always pull before you push**, so you start from everyone's latest.

```
        GitHub (shared remote)
         ↑ push        ↓ pull

you      teammate 1    teammate 2
```

When it goes wrong

Common issues

Table 2: Common snags and their fixes.

Symptom	Cause	Fix
Merge conflict	Two people changed the same lines	Git asks *you* to pick; pull/push often to avoid
"My changes aren't tracked!"	Wrong RStudio **Project** open	Switch to the Project tied to that repo
Push rejected	A teammate pushed first	**Pull**, then push again
File-too-big warning (>50 MB)	Committing a big data file	Don't commit it; add to `.gitignore`

> **Warning**
>
> When truly stuck, Happy Git's troubleshooting chapter and the "Burn it all down" escape hatch (re-clone a fresh copy) will probably help.

What we covered today

- **Why:** version control = full history + revert + safe collaboration; a reproducibility backbone
- **Git vs. GitHub:** the local tool vs. the remote host; a repo is a tracked folder
- **The daily loop:** stage, commit (with a good message), pull, push, mostly from the RStudio Git tab
- **Teams:** one shared repo, everyone clones, *pull before you push*
- **Snags:** merge conflicts, wrong Project, rejected pushes, big files, each has a known fix

Activity

Project Repo Setup Activity

- Activity Canvas Link

Solo: follow Happy Git with R to register on GitHub, install Git, connect it to RStudio, and prove it works with a test repo.

Team: one person creates the project repo and adds the others as collaborators; everyone clones it and pushes at least one commit.

Wrap-up & looking ahead

Upcoming Deadlines

- **Tonight, Fri 6/5, 11:59 p.m.**: HW 3
- **By Tues 6/9, class**: team repo working, **all members able to push** (today's setup guide)
- **Mon 6/8, in class**: in-class join activity (no worksheet to submit) so you can focus on repo setup at home; make sure you attend and are on-time to get credit.

Reach out

Stuck? Office hours, or email me (cgc5478@psu.edu) or Xinyue (xpw5228@psu.edu).

Reference

Syllabus · AI policy · Schedule · Happy Git with R · *DC* Ch. 9 · GitHub